```
        override fun onCreate() {
            // ...

            Hotwire.registerFragmentDestinations(
                // ...
            )

»           Hotwire.registerBridgeComponents(
»               BridgeComponentFactory("button", ::ButtonComponent)
»           )
        }
    }
```

And yes, those two leading colons are intentional! That's Kotlin's way of referencing a class, like in Swift when we used `ButtonComponent.self`.

You'll use Compose to render the contents of the component. And just like in *When to Go Native*, you'll do that with a `ComposeView`. But each `ComposeView` needs a hook to bridge the gap between Jetpack Compose and the traditional Android View system.

We'll use the `HotwireFragment` from Hotwire Native as our bridge. And we can grab a reference to it by casting `BridgeDelegate` in `ButtonComponent`.

ch08_04/android/app/src/main/java/com/masilotti/hikingjournal/components/ButtonComponent.kt

```
    package com.masilotti.hikingjournal.components

    import dev.hotwire.core.bridge.BridgeComponent
    import dev.hotwire.core.bridge.BridgeDelegate
    import dev.hotwire.core.bridge.Message
    import dev.hotwire.navigation.destinations.HotwireDestination
»   import dev.hotwire.navigation.fragments.HotwireFragment

    class ButtonComponent(
        name: String,
        private val bridgeDelegate: BridgeDelegate<HotwireDestination>
    ) : BridgeComponent<HotwireDestination>(name, bridgeDelegate) {
»       private val fragment: HotwireFragment
»           get() = bridgeDelegate.destination.fragment as HotwireFragment
```